

Lecture 03

Data Representations

Comp_Sci 213

Introduction to Computer Systems

Wednesday, Sept. 30, 2026

Kate Smith – Fall 2026

Slides adapted from:

Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

Announcement

- Ignore quizzes on Canvas!
- HW 1 has been posted!

Review: General method for decimal to binary

Continuously divide by 2, keeping track of remainders

Example: convert 26_{10} to binary.

$$26 \div 2 = 13, R = 0$$

$$13 \div 2 = 6, R = 1$$

$$6 \div 2 = 3, R = 0$$

$$3 \div 2 = 1, R = 1$$

$$1 \div 2 = 0, R = 1$$

Digit position of new number moves away from decimal point as number of remainders increases!! (Start with least significant digit, end with most significant)

Result: 0b11010

$$\text{Check: } 1 * 2^4 + 1 * 2^3 + 1 * 2^1 = 26_{10}$$

Topics

- Memory
- Encoding
- Integer Encodings

Topics

- **Memory**
- Encoding
- Integer Encodings

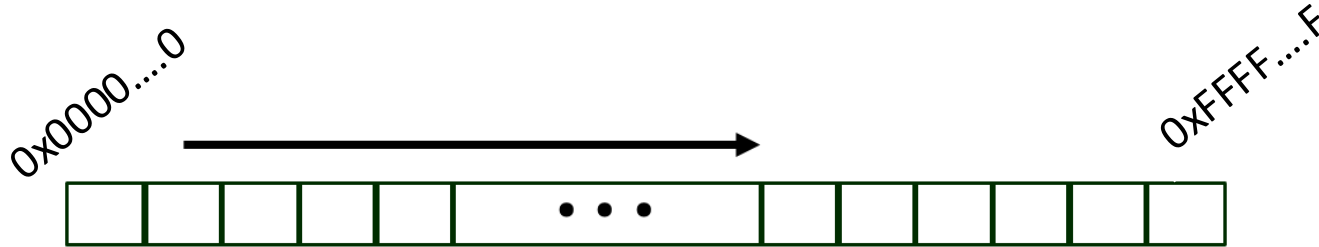
Bytes

- A single bit doesn't hold much information
 - Two values: 0 and 1
- We organize bits (b) in groups of 8, referring to them as **bytes** (B)
 - 8 bits = 1 byte = hexits (hex digits)
- Some examples
 - 1 byte: 0b01100111 = 0x67
 - 2 bytes: 11000100 00101111₂ = 0xC42F

"0b" prefix = it's in binary
"0x" prefix = it's in hex

Byte-oriented memory organization

- Sequences of bits can express numbers
 - Variables are almost always some multiple of number of bytes (i.e. 1 byte, 2 bytes, 8 bytes, etc.)
 - Context is important! Is the value signed, unsigned, etc.
- In a computer system, bytes can be stored in memory
 - Memory is a large array of bytes
 - Each byte has its own address (i.e. a pointer)

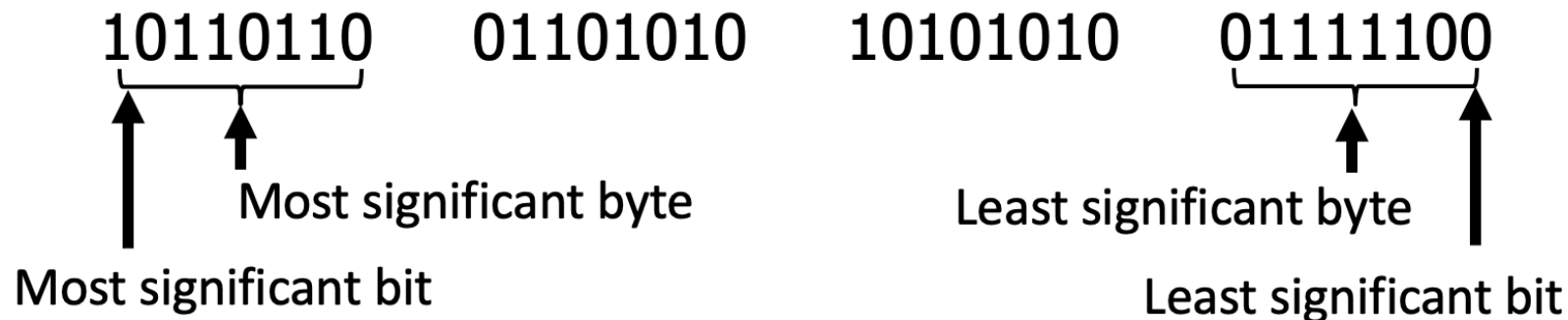


- Compiler and run time system control allocation
 - Where different program objects should be stored
 - Multiple mechanisms, each with its own region: static, stack, and heap

Most/least significant bits/bytes

When working with sequences of bits (or sequences of bytes), we need to be able to talk about specific ones

- ***Most Significant bit (MSb)*** and ***Most Significant Byte (MSB)***
 - Have the largest possible contribution to numeric value
 - Leftmost when writing out the binary sequence
- ***Least Significant bit (LSb)*** and ***Least Significant Byte (LSB)***
 - Have the smallest possible contribution to numeric value
 - Rightmost when writing out the binary sequence



Addressing and byte ordering

For data that spans multiple bytes, we must agree on two things:

1. What should the address of the object be?

- Each byte has its own location
- Once we have fixed an address, how do we find relevant bytes that are all part of a sequence that are united under some context?

2. How should we order the bytes in memory?

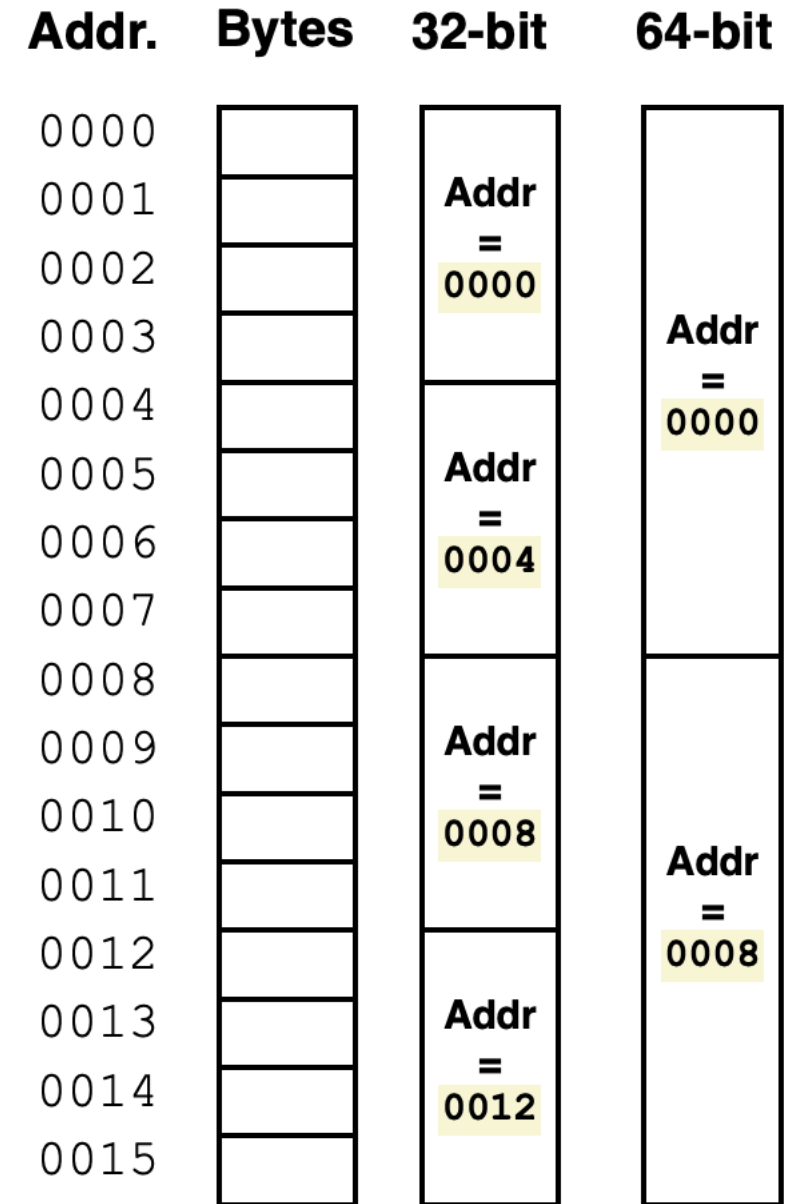
- Do we put the MSB or LSB first?

Multiple answers are possible

- Byte ordering can vary across systems – you must know the specifics of the system you are using!
 - There are tradeoffs associated with what ordering is “correct,” making context critical – more about this later!
- Different byte ordering is fine across different systems, but all parts within a given system must agree

Addressing data in memory

- All addresses refer to one or more bytes, never bits!
- For multi-byte object, the lowest address refers to the entire object
 - Addresses of successive object differ by 4 (32-bit) or 8 (64-bit)
- Systems (almost) universally use the address of the first byte as the address of the whole object

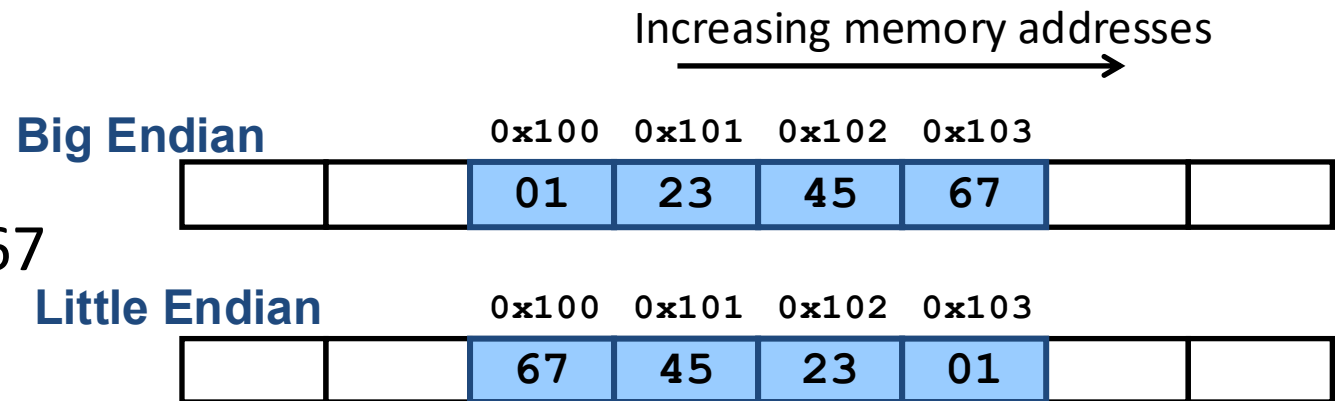


Byte ordering

- When working with data larger than a byte (i.e. a multi-byte object), we need to be concerned about how the bytes are ordered in memory
- Conventions
 - **Big Endian:** seen in computer networks and various other historical computer systems. ***Most significant byte has lowest address / comes first!***
 - **Little Endian:** Intel (x86, x86-64), ARM, and other modern computer systems. ***Least significant byte has lowest address / comes first!***

Example:

- A 4-byte piece of data: 0x01234567
- Address of that data is 0x100



Practice: Reading memory

Assume the memory is ***Little Endian*** (LSB comes first!)

1. What is the four-byte value at 0x2010?
2. What is the two-byte value at 0x2014?
3. What is the one-byte value at 0x2016?

Address	Value
0x2010	0x37
0x2011	0x1A
0x2012	0xBE
0x2013	0x98
0x2014	0x0C
0x2015	0x80
0x2016	0x42

Practice: Reading memory

Assume the memory is Little Endian (LSB comes first!)

1. What is the four-byte value at 0x2010?

0x98BE1A37

2. What is the two-byte value at 0x2014?

0x800C

3. What is the one-byte value at 0x2016?

0x42

Address	Value
0x2010	0x37
0x2011	0x1A
0x2012	0xBE
0x2013	0x98
0x2014	0x0C
0x2015	0x80
0x2016	0x42

Practice: Reading memory

Assume the memory is **Big Endian** (MSB comes first!)

1. What is the four-byte value at 0x2010?
2. What is the two-byte value at 0x2014?
3. What is the one-byte value at 0x2016?

Address	Value
0x2010	0x37
0x2011	0x1A
0x2012	0xBE
0x2013	0x98
0x2014	0x0C
0x2015	0x80
0x2016	0x42

Practice: Reading memory

*Note: endianness
doesn't affect one-byte
values!*

Assume the memory is **Big Endian** (MSB comes first!)

1. What is the four-byte value at 0x2010?

0x371ABE98

2. What is the two-byte value at 0x2014?

0x0C80

3. What is the one-byte value at 0x2016?

0x42

Address	Value
0x2010	0x37
0x2011	0x1A
0x2012	0xBE
0x2013	0x98
0x2014	0x0C
0x2015	0x80
0x2016	0x42

Notes about endianness

- Most Intel-compatible machines operate in little endian
- Most IBM and Oracle machines (because of an acquisition of Sun Microsystems in 2010) operate in big-endian
- Some recent microprocessor chips are bi-endian, so they can operate as either little- or big- endian machines
 - Byte ordering becomes fixed when an OS is selected
- Tradeoffs:
 - Big Endian: matches how humans write left to right, quick sign/magnitude checks, used as the standard in network protocols like TCP/IP, less intuitive for pointer casting / truncation
 - Little Endian: easier type casting (e.g. changing the apparent size of an integer from 4 bytes to 2 bytes), historically helped smaller processors (8-bit or 16-bit) start arithmetic faster because lowest byte was fetched first, massively adopted by industry, counterintuitive for humans while debugging

Tables of memory

Address	0	1	2	3	4	5	6	7
0x1040	2E	E2	BD	62	EF	A0	CD	93
0x1048	A4	75	61	2F	0F	DB	64	A4
0x1050	54	7A	F2	60	6E	47	B0	92
0x1058	DA	72	8F	A8	E5	15	18	CE
0x1060	86	BF	6A	6A	92	99	CF	6C

- Method of displaying large chunks of memory
 - 8 bytes per row (in this example)
 - Data values in hexadecimal
- Memory addresses labeled on the left and byte offset on the top
 - Note: these are in hexadecimal!

Tables of memory

Address	0	1	2	3	4	5	6	7
0x1040	2E	E2	BD	62	EF	A0	CD	93
0x1048	A4	75	61	2F	0F	DB	64	A4
0x1050	54	7A	F2	60	6E	47	B0	92
0x1058	DA	72	8F	A8	E5	15	18	CE
0x1060	86	BF	6A	6A	92	99	CF	6C

- To find an address: deconstruct into:
 - base address (left) + offset (top)
- Remember: addresses are in hexadecimal!
 - $0x1048 + 2 = 0x104A$

Tables of memory

Address	0	1	2	3	4	5	6	7
0x1040	2E	E2	BD	62	EF	A0	CD	93
0x1048	A4	75	61	2F	0F	DB	64	A4
0x1050	54	7A	F2	60	6E	47	B0	92
0x1058	DA	72	8F	A8	E5	15	18	CE
0x1060	86	BF	6A	6A	92	99	CF	6C

- What value is at address 0x1050?
- What value is at address 0x105F?

Tables of memory

Address	0	1	2	3	4	5	6	7
0x1040	2E	E2	BD	62	EF	A0	CD	93
0x1048	A4	75	61	2F	0F	DB	64	A4
0x1050	54	7A	F2	60	6E	47	B0	92
0x1058	DA	72	8F	A8	E5	15	18	CE
0x1060	86	BF	6A	6A	92	99	CF	6C

- What value is at address 0x1050?
 - 0x54 (0x1050 + 0)
- What value is at address 0x105F?
 - 0xCE (0x1058 + 7)

Topics

- Memory
- **Encoding**
- Integer Encodings

Key Idea: What do bits and bytes mean in a system?

- The answer is: it depends!
- Depending on the context, the bits **11000011** could mean
 - The number 195
 - The number -61
 - The number -19/16
 - The character '┘'
 - The **ret** x86 instruction
- You have to know the context to make sense of any bits you have!
 - Looking at the same bits in different contexts can lead to interesting results
 - Information = bits + context!
- An **encoding** is a set of rules that gives meaning / context to bits

Example encoding: ASCII Characters

- **ASCII** = American Standard Code for Information Interchange
 - Standard dating from the 60s
- Maps 8-bit* bit patterns to characters
 - (* the standard is actually 7-bit, leaving the 8th bit unused)
 - Each bit pattern maps to a single character
- Examples
 - $0100\ 0001_2 = 0x41 = 65_{10} = \text{'A'}$
 - $0100\ 0010_2 = 0x42 = 66_{10} = \text{'B'}$
 - $0011\ 0000_2 = 0x30 = 48_{10} = \text{'0'}$
 - $0011\ 0001_2 = 0x31 = 49_{10} = \text{'1'}$
- Reference: <https://www.asciitable.com/>

Full ASCII table

Values listed in:

Decimal

Hexadecimal

Octal

HTML

Character

**You don't need to know the ASCII table. Just understand the concept.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

The Unicode standard

- The ASCII character set is suitable for English-language documents, but it does not include special characters
 - Ç, é, ñ, ©, non-Latin alphabets, characters for the Greek, Russian, and Chinese languages, etc.
- The Unicode Consortium devised the most comprehensive and widely accepted standard for encoding text
 - 100,000+ characters!
 - Even characters for ancient languages of Egypt and Babylon are supported
- For more information, check out <https://home.unicode.org/>
- “Universal Character Set” of Unicode uses a 32-bit representation
 - Alternative coding possible that uses 1 or 2 bytes
 - UTF-8 encodes each character in a sequence of bytes and matches ASCII encoding

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+2B0x	↪	↩	↪	↩	↔	←	↑	↓	↗	↖	↘	↙	↔	↕	↗	↖
U+2B1x	↵	↶	▣	▤	▥	▦	◀	◁	◊	◈	◉	◊	◈	◉	◊	◈
U+2B2x	◐	◑	◒	◓	●	◆	◇	◇	◇	◇	◇	◇	◐	◑	◒	◓
U+2B3x	↔	↔	⊕	⚡	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔
U+2B4x	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔
U+2B5x	★	★	★	★	★	★	★	★	★	★	★	★	★	★	★	★
U+2B6x	←	↑	→	↓	↔	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
U+2B7x	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
U+2B8x	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
U+2B9x	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BAx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BBx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BCx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BDx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BEx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩
U+2BFx	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩	↪	↩

Notes

1.^As of Unicode version 18.0

2.^Grey areas indicate non-assigned code points

Chart credits: https://en.wikipedia.org/wiki/Miscellaneous_Symbols_and_Arrows

Encodings are determined...by people!

- There's no inherent truth in the design of an encoding
 - Encodings are developed for various use cases
 - For example: ASCII encodings put letters in alphabetical order
- Entirely new ways are encoding characters
 - Adoption becomes a challenge with new encodings

What do we need encodings for?

- Numbers
 - Signed and unsigned values
 - Real numbers and decimals
 - Imaginary numbers
 - Mathematical symbols (e.g. ∞ and π)
- Languages
 - Characters in various languages Ω Иسش北
- Colors, user actions, control signals, anything!

Topics

- Memory
- Encoding
- **Integer Encodings**

Integer types in C

- C types describe both encoding and size
- Encoding
 - **signed** or **unsigned**
 - Defaults to signed if no label included
- Sizes
 - 1 byte (8-bit): **char**
 - 2 bytes (16-bit): **short**
 - 4 bytes (32-bit): **int**
 - 8 bytes (64-bit): **long**

Examples:

- **unsigned int**
 - 32-bit unsigned integer
- **signed short**
 - 16-bit signed integer
- **char**
 - 8-bit signed integer

Sizes of C types are technically system dependent

- C only defines types as a “minimum” size
 - They could be different on different computers
 - This is not a good choice
 - You should assume the x86-64 rules
 - Memorize these! Matches prior slide
- How I program
 - Use fixed width integer types `<stdint.h>`
 - `int8_t`, `int16_t`, `int32_t`
 - `uint8_t`, `uint16_t`, `uint32_t`

C Data Type	x86-64 (Bytes)	C Standard* (C99)
char	1	≥ 1
short	2	≥ 2
int	4	≥ 2
long	8	≥ 4
long long	8	≥ 8
float	4	
double	8	
pointer	8	Widths for data, code pointers may differ!

CS213 uses this



Review: pointers

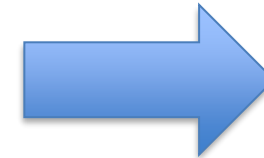
A ***pointer*** is an address and a type telling C how to use that address

```
#include <stdio.h>
int main(void) {

    int x = 42; // Get the address of x
    int *p = &x;

    printf("x = %d\n", x);
    printf("Address of x = %p\n", (void *)&x);
    printf("p = %p\n", (void *)p);
    printf("Value pointed to by p = %d\n", *p);

    return 0;
}
```



Output:

x = 42

Address of x = 0x16f3e...

p = 0x16f3e...

Value pointed to by p = 42

***Note: The (void *) cast is the conventional way to pass a pointer to printf for %p*

Expressing C types in bits

- Two families of encodings to express integers using bits
 - ***Unsigned*** encoding for unsigned integers
 - ***Two's complement*** encoding for signed integers
- Each encoding will use a fixed size (# of bits)
 - For a given machine
 - Size + encoding family determine which C type we're representing
 - Reverse is also true: a C type specifies both size and encoding
 - Fixed size is because computers are finite!

Unsigned integer encoding

- Just write out the number in binary
 - Works for 0 and all positive integers
- Example: encode 104_{10} as an **unsigned** 8-bit integer
 - $104_{10} = 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

$\Rightarrow$ **0b01101000**

$\Rightarrow$ **0x68**

Formal Definition

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

(Binary To Unsigned)

Bounds of unsigned integers

- For a fixed width **w**, a limited range of integers can be expressed
 - Smallest value (we will call **UMin**):
 - all 0s bit pattern: 000...0, value of 0
 - Largest value (we will call **UMax**):
 - all 1s bit pattern: 111...1, value of $2^w - 1$
 - $2^w - 1 = 1 \times 2^{w-1} + 1 \times 2^{w-2} + \dots + 1 \times 2^1 + 1 \times 2^0 = 11111\dots$
- Maximum 8-bit number = $2^8 - 1 = 256 - 1 = 255$

Encoding signed integers

- What's different about representing a signed number?
 - It can be negative!
- So, we're going to have to somehow represent values that are negative and positive
- There are actually many different encodings capable of doing this
 - This is when that “nice encoding” versus “annoying encoding” matters
 - We'll focus on the most-commonly used method: two's complement

Doesn't work: attempting signed encoding

- Goal: encode integers that can be positive or negative
- First attempt: we can use the most significant bit for sign
 - “Sign-and-magnitude” encoding
 - In 8-bits:

• +4 = 00000100	+127 = 01111111	+0 = 00000000
• -4 = 10000100	-127 = 11111111	-0 = 10000000
- Annoying problem: we have two representations of zero!
- Also annoying: hardware to do math with signed and unsigned numbers gets complicated...

Two's complement encoding

- Bad news: need to make the encoding more complicated to handle negatives
- Plan:
 - Start with unsigned encoding, but then make ONLY the largest power negative
 - Example: for 8 bits, most significant bit is worth -2^7 not $+2^7$ (other bits are still positive)
- To encode a negative integer
 - First, set the most significant bit to 1 to start with a big negative number
 - Then, add positive powers of 2 (the other bits) to “get back” to number we want
- Example: encode -6 as a 4-bit two's complement integer
 - $-6_{10} = 1 \times -2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \Rightarrow 0b1010 \Rightarrow 0xa$

Two's complement examples

- Encode -100 as an 8-bit two's complement number

- $-100_{10} = 1 \times -2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

$$\begin{array}{cccccccc} -128 & +0 & +0 & +16 & +8 & +4 & +0 & +0 \end{array}$$

Problem becomes:

encode +28 as a 7-bit unsigned number

- $-100_{10} = 0b10011100 = 0x9C$
- **Shortcut:** determine positive version of number, flip all the bits, and add one
 - $100_{10} = 0b01100100$
 - Flipped = $0b10011011$
 - Plus 1 = $0b10011100 = 0x9C$ We'll talk about binary addition next lecture

Interpreting binary signed values

Formal Definition

- Converting binary to signed:

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Sign bit

- Note: most significant bit tells us sign!! 1-> negative
 - Checking if a number is negative is just checking that top bit
- Only one encoding exists for zero – that's helpful!
 - 0b00000000 = 0 0b10000000 = -128
- -1: 0b111...1 = -1 (regardless of number of bits!)

Bounds of two's complement integers

- For a fixed width **w**, a limited range of integers can be expressed
 - Smallest value, most negative (we will call **TMin**):
 - 1 followed by all 0s bit pattern: $100\dots0 = -2^{w-1}$
 - Largest value, most positive (we will call **TMax**):
 - 0 followed by all 1s bit pattern: $01\dots1$, value of $2^{w-1} - 1$
- Beware the asymmetry! Bigger negative number than positive

Ranges for different bit amounts

	W			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
Umin	0	0	0	0
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

- Observations

- $|TMin| = TMax + 1$
 - Asymmetric range
- $Umax = 2 * TMax + 1$

- C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- Values are platform specific

Unsigned & Signed Numeric Values

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

- **Equivalence**
 - Same encodings for non-negative values
- **Uniqueness**
 - Every bit pattern represents a single, unique integer value
 - Each representable integer has a single, unique bit encoding
- **⇒ Can Invert Mappings**
 - Can go from bits to number and back, and vice versa

Practice

What range of integers can be represented with 5-bit two's complement?

- A. -31 to +31
- B. -15 to +15
- C. 0 to +31
- D. -16 to +15
- E. -32 to +31

Practice

What range of integers can be represented with 5-bit two's complement?

- A. -31 to +31 No asymmetry & 6-bits
- B. -15 to +15 No asymmetry
- C. 0 to +31 Unsigned
- D. -16 to +15 Correct
- E. -32 to +31 6-bits

Casting signed to unsigned

- C allows conversions from signed to unsigned (and vice versa)

```
short int      x = 15213;
unsigned short int ux = (unsigned short) x;
short int      y = -15213;
unsigned short int uy = y; /* implicit cast! */
```

- Resulting value
 - Not based on current decimal value: instead keep the bits and **reinterpret** them!
 - Non-negative values unchanged
 - $ux = 15213$
 - Negative values change into (large) positive values (and vice versa)
 - $uy = 50323$
- Warning: Casts can be implicit in assignments or function calls!

Mapping Signed $\leftrightarrow$ Unsigned (4 bits)

Large negative
value becomes
large positive!

Bits	Signed		Unsigned
0000	0	$\longleftrightarrow$ =	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	$\xrightarrow{+16 \text{ (i.e., } 2^4)}$ $\xleftarrow{-16 \text{ (i.e., } 2^4)}$	8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

Example

- Convert signed 8-bit number -120 into an unsigned 8-bit number

1. Convert -120 into binary

2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number
 1. Convert -120 into binary
 $-120 = -128 + 8 =$
 2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number

1. Convert -120 into binary

$$-120 = -128 + 8 =$$

$$1 \times (-128) + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

0b 1000 1000

2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number

1. Convert -120 into binary

$$-120 = -128 + 8 =$$

$$1 \times (-128) + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

$$0b\ 1000\ 1000$$

2. Convert binary back into unsigned decimal

$$1 \times 128 + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

$$128 + 8 = 136$$

Why convert bit length?

- Hardware has physical limits
 - We cannot hold an infinite number of bits
- So sometimes we have to change the sizes of values
 - Reducing to use fewer bits: Truncation
 - Expanding to use more bits: Extension
 - Computer hardware automatically follows these rules
 - So programming languages automatically do them as well
- These aren't mathematical operations, they're hardware limitations
 - So they might not work as you expected

Truncation (reducing number of bits)

- Truncation rule:
 - Just delete most-significant bits until the value is the correct size
 - Same rule no matter the encoding
- If the value fits in both bit sizes, it remains the same
- If not, it will be totally different
 - Result becomes identical to modular arithmetic
 - But we can always just do the truncation and check result
- 6-bit to 2-bit truncation
 - $0b000010 \rightarrow 0b\text{~~0000~~10} = 0b10$ (decimal 2 becomes decimal 2)
 - $0b110010 \rightarrow 0b\text{~~1100~~10} = 0b10$ (decimal 50 becomes decimal 2)
Identical to 50 modulo 4 = 2

Aside: Why do truncation?

- It can totally mess up the value!! So why do it?
- Because you sometimes have to
 - You might have a 32-bit result, but only be able to store an 8-bit value
 - Good code checks whether the result is in-bounds, and takes some action if it isn't
 - Maybe just set to maximum
 - Maybe throw an error

Extension (increasing number of bits)

- Extension rule:
 - Just add most-significant bits until the value is the correct size
- But should we add zeros or ones??
 - Unsigned extension: add zeros
 - Signed extension: add copies of most-significant bit (0 or 1)
 - Both results preserve the original value!
- 4-bit to 7-bit extension
 - Unsigned 0b0110 -> 0b0000110 (decimal 6 stays decimal 6)
 - Unsigned 0b1000 -> 0b0001000 (decimal 8 stays decimal 8)
 - Signed 0b0110 -> 0b0000110 (decimal 6 stays decimal 6)
 - Signed 0b1000 -> 0b1111000 (decimal -8 stays decimal -8 = -64+32+16+8)

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
 - Process: truncate extra bits
 - Answer is **0x27**
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
 - Process: truncate extra bits
 - Answer is **0x27**
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)
 - Process: sign extend. Is the most-significant bit one? Yes!
 - Answer is **0xFFFF0**

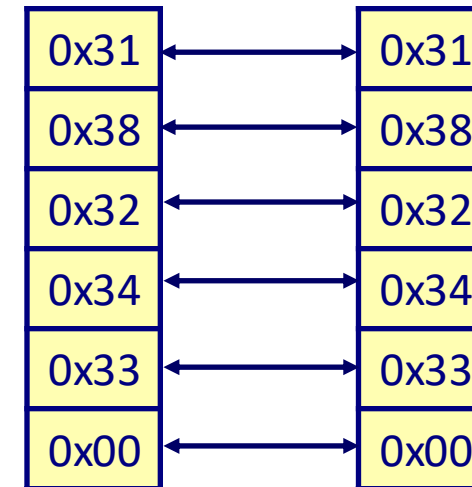
Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Encoding strings (The C way)

- Represented by array of characters
 - Each character encoded in ASCII format
 - NULL character (code 0) to mark the end
- Compatibility
 - Byte ordering not an issue (data all single-byte!)
 - ASCII text files generally platform independent
 - Except for different conventions of line termination character(s)!

```
char S[6] = "18243";
```

Big-Endian Little-Endian



Encoding color

- RGB colors
 - 3-byte values
 - First byte is Red, then Green, then Blue
- Usually specified in hexadecimal
 - #FF0000 -> maximum red, zero green or blue
 - #4E2A84 -> 1/4 red, 1/8 blue, 1/2 green (Northwestern Purple)
- 2^{24} possible colors = 16777216 colors

